

國立台北科技大學

NATIONAL TAIPEI UNIVERSITY OF TECHNOLOGY

電機系實習報告

課程名稱：微處理機實習

作業名稱：LAB10

組別：第 13 組

姓名及學號：

張博翔 b812111001

孫皓 110350450

中 華 民 國 114 年 11 月 24 日

作業基本題 1 (40%)

****基本題 1 (40%)：** 修改範例 ch 6_2，使進一步滿足以下需求：

KTduino 積極體驗版提供四個 RGB LED，而原範例只使用其中一個 (DS10A)，若要四個一起亮(相同任意顏色即可)，如何修改？

實驗結果拍攝： 拍攝四個 RGB LED 一起亮相同顏色的照片 1 張。

(1) 電路設計

LED P10-5 之紅色接腳連接在 Pin 6

LEDP 10-6 之綠色接腳連接在 Pin 5

LED P10-7 之藍色接腳連接在 Pin 3, P10-1 接地

(2) 程式碼

```
(3)  /*
(4)   範例 ch6_2
(5)   LED 之紅色接腳連接在 Pin 6
(6)   LED 之綠色接腳連接在 Pin 5
(7)   LED 之藍色接腳連接在 Pin 3, P10-1 接地
(8)  */
(9)  const int RGB[3] = {6,5,3};    // 宣告 LED 顏色接腳位置
(10) const int LED[4] = {8,9,10,11};
(11) void RGB_LED(int R, int G, int B); // 宣告 RGB_LED 函數(自建函數)
(12) // 初始設定
(13) void setup() {
(14)     for(int i=0;i<3;i++){
(15)         pinMode(RGB[i], OUTPUT);
```

```

(16)     }
(17)     for(int i=0;i<4;i++){
(18)         pinMode(LED[i], OUTPUT);
(19)         digitalWrite(LED[i],LOW);
(20)     }
(21) }
(22) // 主程式
(23) void loop() {
(24)     RGB_LED(255, 0, 0);    // 狀態 1
(25)     delay(1000);          // 持續 1 秒
(26)     RGB_LED( 0,255, 0);    // 狀態 2
(27)     delay(1000);          // 持續 1 秒
(28)     RGB_LED( 0, 0,255);    // 狀態 3
(29)     delay(1000);          // 持續 1 秒
(30)     RGB_LED(255,255, 0);   // 狀態 4
(31)     delay(1000);          // 持續 1 秒
(32)     RGB_LED( 0,255,255);   // 狀態 5
(33)     delay(1000);          // 持續 1 秒
(34)     RGB_LED(255, 0,255);   // 狀態 6
(35)     delay(1000);          // 持續 1 秒
(36)     RGB_LED(255,255,255);  // 狀態 7
(37)     delay(1000);          // 持續 1 秒
(38)     for(int i=0;i<256;i++) // 藍色從 0 到 255 變化
(39)         for(int j=0;j<256;j++) // 綠色從 0 到 255 變化
(40)             for(int k=0;k<256;k++) { // 紅色從 0 到 255 變化
(41)                 RGB_LED(k,j,i);    // 驅動 RGB LED
(42)                 delay(10);          // 持續 0.05 秒
(43)             }
(44) }
(45) // RGB LED 驅動函數
(46) void RGB_LED(int R, int G, int B) {
(47)     analogWrite(RGB[0], R);    // 輸出紅色信號(PWM)
(48)     analogWrite(RGB[1], G);    // 輸出綠色信號(PWM)
(49)     analogWrite(RGB[2], B);    // 輸出藍色信號(PWM)
(50) }
(51)

```

(3)實驗結果

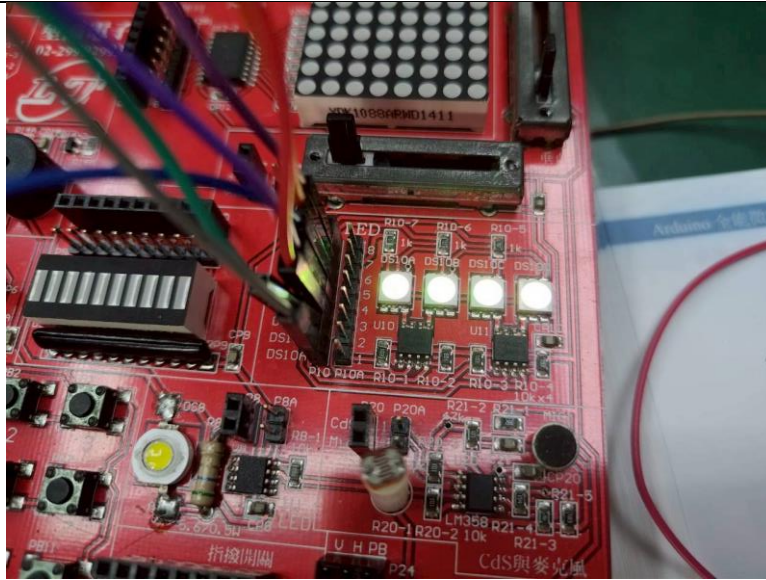


圖 1

****基本題 2 (40%):**

修改範例 ch 6_3，使進一步滿足以下需求：

在原單元裡，顏色的移動方向是由左向右移，請將移動方向改為由右往左移，並改變每個 LED 的亮燈時間由 2ms 改為 5ms。

實驗結果拍攝：連續拍 4 張照片以顯示 LED 顏色由右向左移的結果。

(1) 電路設計

P LED P10-5 之紅色接腳連接在 Pin 6

LEDP 10-6 之綠色接腳連接在 Pin 5

LED P10-7 之藍色接腳連接在 Pin 3, P10-1 接地

P10-1~P10-4 接到 Arduino 8~11 腳位

(2)程式碼

```
/*
範例 ch6_3
LED 之紅色接腳連接在 Pin 6
LED 之綠色接腳連接在 Pin 5
LED 之藍色接腳連接在 Pin 3
*/
const int RGB[3] = {6,5,3};    // 宣告 LED 顏色接腳位置
const int scan[4] = {11,10,9,8}; // 宣告 LED 共同點位置(低態掃描)
int PP ;                        // 宣告掃描指標變數
void RGB_LED1(int R, int G, int B, int duration); // 宣告 RGB_LED 函數
// R:紅色指標，G:綠色指標，B:藍色指標，duration:持續顯示時間)
// 初始設定
```

```

void setup() {
    for(int i=0;i<3;i++) pinMode(RED[i], OUTPUT);
    for(int i=0;i<4;i++) pinMode(scan[i], OUTPUT);
    for(int i=0;i<4;i++) digitalWrite(scan[i], 1); // 關閉掃描線
}
// 主程式
void loop() {
    for(int Start=3; Start>-1; Start--) { // 四次移位
        for(int i=0; i<50; i++) { // 重複掃描 50 次(每次 8ms, 0.4 秒)
            // 第一條掃描線
            digitalWrite(scan[Start], 0); // 輸出第一條掃描線
            RGB_LED1(255, 0, 255, 2); // 紅色 2ms
            digitalWrite(scan[Start], 1); // 關閉掃描線
            // 第二條掃描線
            PP=Start+1; // 第二條掃描線之指標
            if (PP>3) PP-=4; // 若指標超出範圍, 則減 4
            digitalWrite(scan[PP], 0); // 輸出第二條掃描線
            RGB_LED1(255, 255, 0, 2); // 綠色 2ms
            digitalWrite(scan[PP], 1); // 關閉掃描線
            // 第三條掃描線
            PP=Start+2; // 第三條掃描線之指標
            if (PP>3) PP-=4; // 若指標超出範圍, 則減 4
            digitalWrite(scan[PP], 0); // 輸出第三條掃描線
            RGB_LED1(0, 255, 255, 2); // 藍色 2ms
            digitalWrite(scan[PP], 1); // 關閉掃描線
            // 第四條掃描線
            PP=Start+3; // 第四條掃描線之指標
            if (PP>3) PP-=4; // 若指標超出範圍, 則減 4
            digitalWrite(scan[PP], 0); // 輸出第四條掃描線
            RGB_LED1(255, 255, 255, 2); // 白色 2ms
            digitalWrite(scan[PP], 1); // 關閉掃描線
        }
    }
}
// RGB LED 驅動函數
void RGB_LED1(int R, int G, int B, int duration) {
    analogWrite(RED[0], R); // 輸出紅色信號(PWM)
    analogWrite(RED[1], G); // 輸出綠色信號(PWM)
    analogWrite(RED[2], B); // 輸出藍色信號(PWM)
    delay(duration); // 持續顯示時間
}

```

(3)實驗結果



圖 1

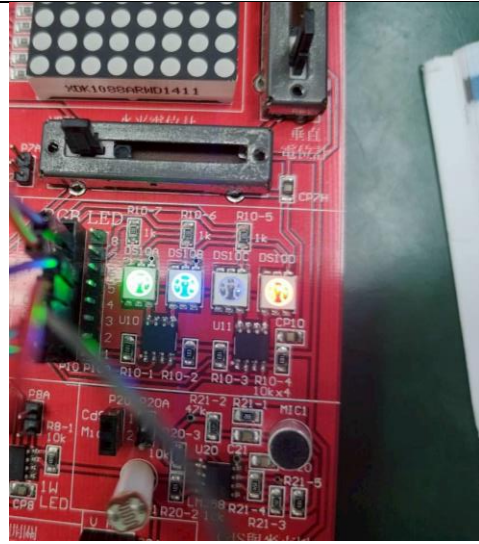


圖 2

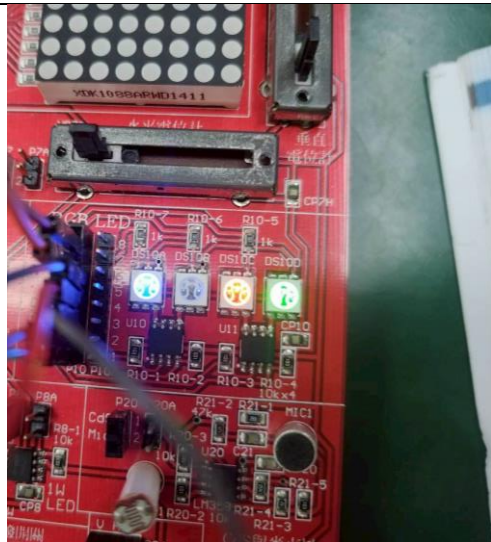


圖 3

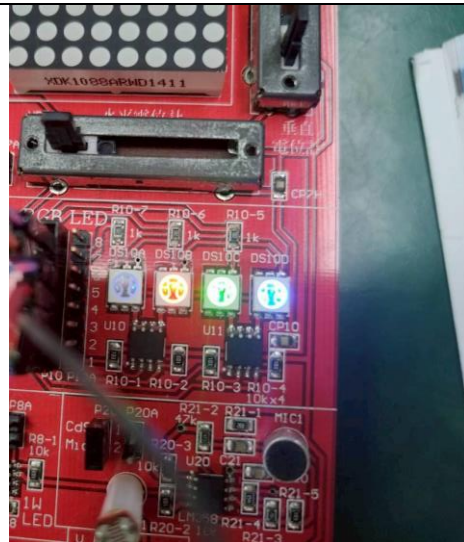


圖 4

**加分題 1 (20%)

請設計一個兩津勘吉的眉毛顏色，由中間往兩邊移，再由兩邊往中間移。

(hint: 會用到下列指令:

```
clearLEDs();          // 關閉所有 LED

leds.setPixelColor(LED_CNT/2 - i, color); // 左邊

leds.setPixelColor(LED_CNT/2 + i, color); // 右邊

leds.show();          // 開啟所有 LED

delay(wait);

)
```

實驗結果拍攝: 連續拍 4 張照片，2 張顯示由中間往兩邊移，2 張顯示由兩邊往中間移。

(1) 電路設計

WS2812 系列串列式 RGB LED 連接在 Pin 2

(2) 程式碼

```
/*
  範例 ch6_5
  WS2812 系列串列式 RGB LED 連接在 Pin 2
*/
#include <Adafruit_NeoPixel.h>    // 包含 Adafruit_NeoPixel 標頭檔
#define TOP_DOWN 0
#define DOWN_TOP 1
#define MEDIUMSPRINGGREEN 0x00FA9A
#define INDIGO 0x4B0082
```

```

#define PIN 2
#define LED_CNT 20
// 建構名為 leds 的 Adafruit_NeoPixel 類別
Adafruit_NeoPixel leds = Adafruit_NeoPixel(50,2,NEO_GRB + NEO_KHZ800);
//Adafruit_NeoPixel strip = Adafruit_NeoPixel(50,2,NEO_RGB+NEO_KHZ800);
#include "my_NeoPixel.h" // 可直接使用顏色定義標頭檔
int wait = 100;

// 設定=====
void setup() {
    leds.begin(); // 初始化
    clearLEDs(); // 關閉所有 LED
    leds.show(); // 開啟所有 LED
}
// 主程式=====
void loop(){
/* // 漫步彩虹大道 (重複 10 週)
    for (int i=0; i<LED_CNT*10; i++) {
        rainbow(i); // 執行彩虹顯示功能
        leds.setBrightness(20);
        delay(100); // 持續顯示
    }

    // 靛藍龍捲風 (重複 10 週)
    for (int i=0; i<10; i++) {
        cyclone(INDIGO, 500); // 靛藍龍捲風眼
        // 龍捲風函數：第一個引數為顏色，第二個引數為週期(時間)
    }

    // 春雨(執行由上而下瀑布 20 次)
    for (int i=0; i<20; i++) {
        cascade(MEDIUMSPRINGGREEN, TOP_DOWN, 100);
        // 第一個引數為顏色，第二個引數為方向，第二個引數為掉落時間(ms)
    }
*/
    for (int i=0; i<10; i++) {
        clearLEDs(); // 關閉所有 LED
        leds.setBrightness(50);
        leds.setPixelColor(LED_CNT/2 - i, 255,100,255); // 左邊
        leds.setPixelColor(LED_CNT/2 + i-1, 255,100,255); // 右邊

        leds.show(); // 開啟所有 LED
        delay(wait);
    }
    for (int i=10; i>-1; i--) {
        clearLEDs(); // 關閉所有 LED
        leds.setBrightness(50);

        leds.setPixelColor(LED_CNT/2 - i, 255,100,255); // 左邊
        leds.setPixelColor(LED_CNT/2 + i-1, 255,100,255); // 右邊
    }
}

```

```
    leds.show();           // 開啟所有 LED
    delay(wait);
  }
}
```

(3)實驗結果

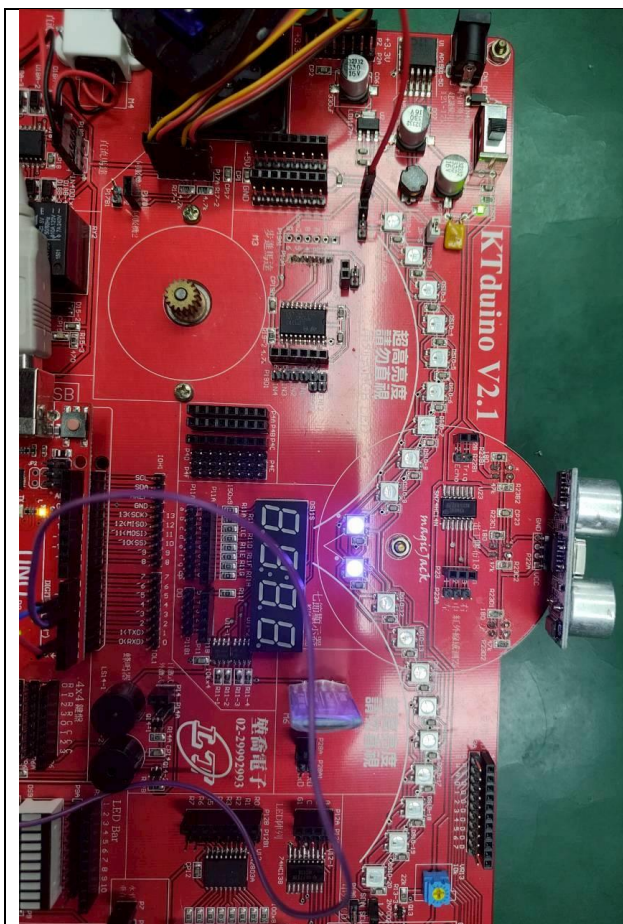


圖 1

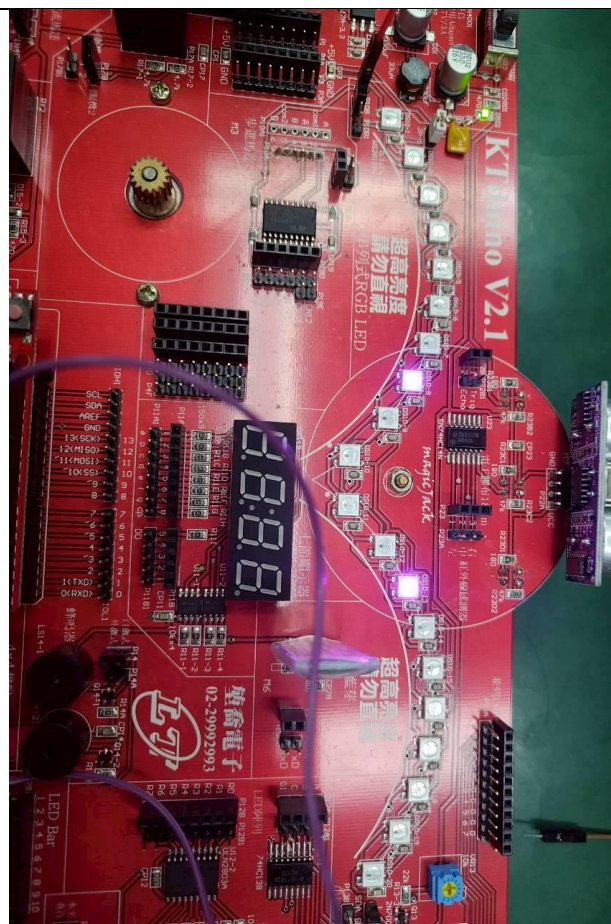


圖 2

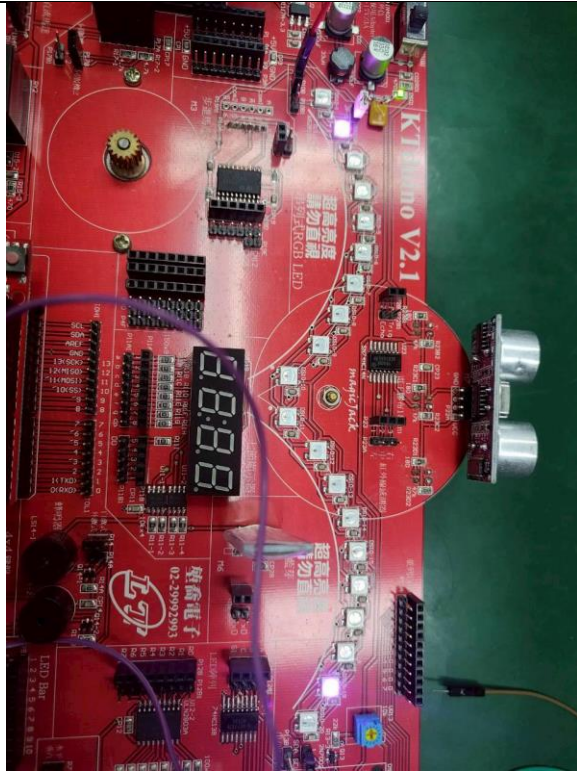


圖 3

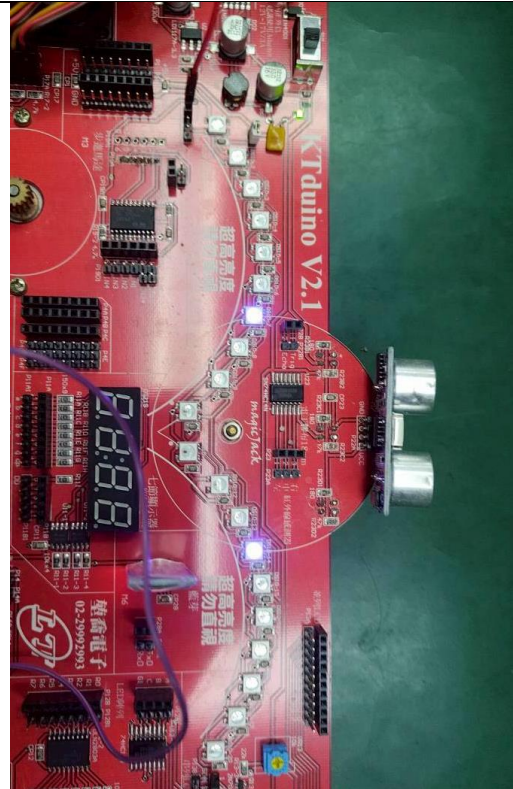


圖 4

心得

張博翔：

這次的實驗主要是在練習 RGB LED 的控制，內容比之前的 LED 或七段顯示器更豐富，因為 RGB 需要同時控制三個通道的亮度，才能組合出不同顏色。基本題 1 是把原本只亮一顆的 RGB LED 改成四顆一起亮，雖然邏輯不難，但讓我更理解腳位控制與函式呼叫的差異。基本題 2 則是修改掃描式控制，把顏色移動方向改成由右往左，並調整顯示時間，這部分讓我再一次體會到掃描顯示的原理，不是每顆同時亮，而是

快速輪流點亮造成連續效果。加分題則是做出兩津勘吉眉毛效果，必須同時從中間往外、再從外往內掃描，過程中需要精準控制索引位置。整體來說，這次實驗讓我更熟悉 RGB LED 的各種控制方式，也對掃描式顯示的概念更加清楚。

孫皓：

本週的重點是 RGB LED 與掃描式顯示，實驗比想像中更有趣。基本題 1 要求四顆 RGB LED 同時亮相同顏色，讓我學到 RGB 其實只是同時操作 R、G、B 三個腳位的輸出值。基本題 2 則要把顏色從右往左移動，而且延長停留時間，讓掃描效果更明顯，這也加深我對亮燈時間與視覺暫留之間關係的理解。加分題的眉毛動畫更有挑戰性，要從中心往外、再從外往中心，像是在控制一組動畫邏輯，讓我感覺像是在製作 LED 特效。這次實驗讓我更熟悉 RGB 控制與動畫設計的方式，也讓我覺得玩 LED 是一件既實用又好玩的事情。